Q No.	Question	Example Input	Example Output
1	Find the length of a string.	S = "programming"	11
2	Copy one string to another.	S1 = "source"	S2 becomes "source"
3	Concatenate two strings.	S1 = "Hello", S2 = "World"	"HelloWorld"
4	Compare two strings (case-sensitive).	S1 = "Test", S2 = "test"	Not Equal (or non-zero value)
5	Compare two strings ignoring case.	S1 = "Test", S2 = "test"	Equal (or 0)
6	Convert a string to uppercase.	S = "hello"	"HELLO"
7	Convert a string to lowercase.	S = "HELLO"	"hello"
8	Toggle the case of each character.	S = "MiXED"	"mIxeD"
9	Check whether a string is empty.	S1 = "", S2 = "A"	S1: True, S2: False
10	Trim leading, trailing, or extra spaces.	S = " hello world "	"hello world"
11	Get the character at a given index.	S = "Python", Index = 2	t'
12	Get the Unicode code point of a character at index.	S = "A", Index = 0	65
13	Get the Unicode code point before index.	S = "Hello", Index = 1	72 (Unicode for 'H')
14	Find the first occurrence of a character.	S = "banana", Char = 'a'	1 (index)
15	Find the last occurrence of a character.	S = "banana", Char = 'a'	5 (index)
16	Count total occurrences of a character.	S = "programming", Char = 'g'	2
17	Remove occurrences of a character.	S = "banana", Char = 'a', Remove All	"bnn"
18	Replace occurrences of a character.	S = "apple", Old='p', New='x'	"axxle"
19	Find the highest frequency character.	S = "abracadabra"	a'
20	Find the lowest frequency character.	S = "aabbcde"	c', 'd', 'e' (any one or all)
21	Find the first non-repeating character.	S = "aabbcde"	c'
22	Find the last repeating character.	S = "abracadabra"	r'
23	Print all characters that occur exactly twice.	S = "aabbcdee"	b', 'e'
24	Check if all characters in a string are unique.	S1 = "abc", S2 = "abca"	S1: True, S2: False
25	Count total words in a string.	S = "This is a test"	4
26	Find the first occurrence of a word.	S = "Test this test", Word = "test"	10 (index)
27	Find the last occurrence of a word.	S = "Test this test", Word = "test"	15 (index)
28	Count occurrences of a word.	S = "word word other word", Word = "word"	3
29	Remove occurrences of a word.	S = "a test b test c", Word = "test", Remove All	"a b c"
30	Replace a word with another word.	S = "old data", Old="old", New="new"	"new data"
31	Remove duplicate words.	S = "the cat and the dog"	"the cat and dog"
32	Count frequency of each word.	S = "apple banana apple"	apple: 2, banana: 1
33	Find the longest word.	S = "find the longest word"	"longest"
34	Find the shortest word.	S = "find the shortest word"	"the"
35	Find the first palindrome word.	S = "this madam is here"	"madam"
36	Reverse order of words.	S = "one two three"	"three two one"
37	Reverse each word.	S = "cat dog"	"tac god"
38	Reverse words without split().	S = "a b c"	"c b a"
39	Search all occurrences of a character.	S = "banana", Char='a'	1, 3, 5 (indices)
40	Search all occurrences of a word.	S = "a b a b", Word='b'	2, 6 (start indices)
41	Check if a string contains a substring (without using built-in method).	S1 = "Hello", Sub="ell"	TRUE
42	Check if two strings are equal without equals().	S1 = "abc", S2 = "abc"	TRUE
43	Check if two strings are rotations of each other.	S1 = "abcde", S2 = "cdeab"	TRUE
44	Check if two strings are anagrams.	S1 = "listen", S2 = "silent"	TRUE
45	Check whether a string starts/ends with another string.	S = "apple pie", Prefix = "apple", Suffix = "pie"	Start: True, End: True
46	Check if a substring appears at both the start and end.	S = "abcabca", Sub="abca"	TRUE

47	Check for substring using concatenation trick.	S1="CDAB", S2="ABCD"	True (S1 is in S2+S2)	
48	Remove all vowels.	S = "aeiou XYZ"	" XYZ"	
49	Replace all consonants with '*' (Example suggests replacing non-vowels).	S = "apple"	"ap*le" (or similar output depending on implementation)	
50	Remove all digits.	S = "a1b2c3"	"abc"	
51	Extract only digits.	S = "a1b2c3"	"123"	
52	Remove all special characters.	S = "a!@b#c"	"abc"	
53	Remove punctuation.	S = "Hello, world!"	"Hello world"	
54	Replace duplicate chars with '$'.	S = "hello"	"he$lo"	
55	Reverse only vowels.	S = "hello"	"holle"	
56	Reverse only consonants.	S = "apple"	"eplpa"	
57	Merge two strings alternatively.	S1 = "ABC", S2 = "def"	"AdBeCf"	
58	Rotate characters left by 2 positions.	S = "abcde"	"cdeab"	
59	Rotate characters right by 3 positions.	S = "abcde"	"cdeab"	
60	Append two strings but remove adjacent duplicates.	S1="miss", S2="issippi"	"misisipi"	
21	Find the first non-repeating character.	S = "aabbcde"	c'	
22	Find the last repeating character.	S = "abracadabra"	r'	
23	Print all characters that occur exactly twice.	S = "aabbcdee"	b', 'e'	
24	Check if all characters in a string are unique (no repetition).	S1 = "abc", S2 = "abca"	S1: True, S2: False	
25	Count total number of words in a string.	S = "This is a test"	4	
26	Find the first occurrence of a word.	S = "Test this test", Word = "test"	10 (index)	
27	Find the last occurrence of a word.	S = "Test this test", Word = "test"	15 (index)	
28	Count occurrences of a word.	S = "word word other word", Word = "word"	3	
29	Remove the first, last, or all occurrences of a word.	S = "a test b test c", Word = "test", Remove All	"a b c"	
30	Replace a word with another word.	S = "old data", Old = "old", New = "new"	"new data"	
31	Remove duplicate words from a string.	S = "the cat and the dog"	"the cat and dog"	
32	Count the frequency of each word.	S = "apple banana apple"	apple: 2, banana: 1	
33	Find the longest word.	S = "find the longest word"	"longest"	
34	Find the shortest word.	S = "find the shortest word"	"the"	
35	Find the first word that is a palindrome.	S = "this madam is here"	"madam"	
36	Reverse the order of words.	S = "one two three"	"three two one"	
37	Reverse each word in a string.	S = "cat dog"	"tac god"	
38	Reverse words without using split().	S = "a b c"	"c b a"	
39	Search all occurrences of a character.	S = "banana", Char = 'a'	1, 3, 5 (indices)	
40	Search all occurrences of a word.	S = "a b a b", Word = "b"	2, 6 (start indices)	
41	Check if a string contains a substring (without using contains()).	S1 = "Hello", Sub = "ell"	TRUE	
42	Check if two strings are equal without using equals().	S1 = "abc", S2 = "abc"	TRUE	
43	Check if two strings are rotations of each other.	S1 = "abcde", S2 = "cdeab"	TRUE	
44	Check if two strings are anagrams.	S1 = "listen", S2 = "silent"	TRUE	
45	Check whether a string starts with or ends with another string.	S = "apple pie", Prefix = "apple", Suffix = "pie"	Start: True, End: True	
46	Check if a substring appears at both the start and end.	S = "abcabca", Sub = "abca"	TRUE	
47	Check if one string is a substring of another using only concatenation.	S1 = "CDAB", S2 = "ABCD"	S1 is substring of S2S2 (ABCDABCD) → True	
48	Remove all vowels.	S = "aeiou XYZ"	" XYZ"	
49	Replace all consonants with ''.	S = "apple"	"ae"	
50	Remove all digits.	S = "a1b2c3"	"abc"	
51	Extract only digits.	S = "a1b2c3"	"123"	
52	Remove all special characters.	S = "a!@b#c"	"abc"	
53	Remove all punctuation characters.	S = "Hello, world!"	"Hello world"	

54	Replace all duplicate characters with '$'.	S = "hello"	"he$lo"	
55	Reverse only vowels.	S = "hello"	"holle"	
56	Reverse only consonants.	S = "apple"	"eplpa"	
57	Merge two strings alternatively (char by char).	S1 = "ABC", S2 = "def"	"AdBeCf"	
58	Rotate characters by 2 positions to the left.	S = "abcde"	"cdeab"	
59	Rotate characters by 3 positions to the right.	S = "abcde"	"cdeab"	
60	Append two strings but remove duplicate adjacent characters.	S1 = "miss", S2 = "issippi"	"misisipi"	
61	Count total alphabets, digits, and special characters.	S = "a1b!c2"	Alphabets: 3, Digits: 2, Special: 1	
62	Count vowels and consonants.	S = "apple"	Vowels: 2, Consonants: 3	
63	Count frequency of each character.	S = "aabcc"	a: 2, b: 1, c: 2	
64	Count frequency of each vowel.	S = "programming"	o: 1, a: 1 (e, i, u: 0)	
65	Count palindromic substrings.	S = "aaa"	6 (a, a, a, aa, aa, aaa)	
66	Count number of sentences in a paragraph.	P = "This. Is. Test."	3	
67	Count how many times a substring appears.	S = "abab", Sub = "ab"	2	
68	Count the sum of digits present in a string.	S = "a1b2c3"	6 (1+2+3)	
69	Count how many times 'life' appears in a string.	S = "life is life"	2	
70	Compare the number of times 'the' and 'is' appear.	S = "the cat is on the mat"	the: 2, is: 1 (theis)	
71	Print all substrings.	S = "abc"	"a, b, c, ab, bc, abc"	
72	Print all substrings of length n.	S = "abc", n = 2	"ab, bc"	
73	Find the longest palindromic substring.	S = "babad"	"bab" (or "aba")	
74	Find the longest substring without repeating characters.	S = "abcabcbb"	"abc"	
75	Find the longest common prefix among strings.	Strings = ["flower", "flow", "flight"]	"fl"	
76	Find the longest common suffix among strings.	Strings = ["baking", "making", "taking"]	"king"	
77	Find the longest substring that appears at both ends.	S = "abracadabra"	"abra"	
78	Find the longest mirror-image substring at both ends.	S = "aabccbaa"	"aab"	
79	Divide a string into n equal parts.	S = "abcdef", n = 3	"ab", "cd", "ef"	
80	Print list items containing all characters of a given word.	List = ["apple", "plea"], Word = "pal"	"apple", "plea"	
81	Generate a hash code or UUID.	S = "test"	Hash: 3556498 (Example hash code)	
82	Create a string from a character array.	Char[] = {'h', 'i'}	"hi"	
83	Create a string from a byte array.	Byte[] = {72, 101, 108} (ASCII for H, e, l)	"Hel"	
84	Print ASCII value of each character.	S = "A"	A: 65	
85	Convert string into a char array without built-in functions.	S = "test"	{'t', 'e', 's', 't'}	
86	Print all permutations of a string without repetition.	S = "ab"	"ab", "ba"	
87	Print all permutations of a string with repetition.	S = "aab"	"aab", "aba", "baa"	
88	Rearrange a string so that identical characters are at least d distance apart.	S = "aaabc", d = 2	"abaca"	
89	Remove 'b' and 'ac' from a string.	S = "abacbb"	"c"	
90	Remove adjacent duplicates recursively.	S = "azxxzy"	"ay"	
91	Check if two strings are interleaving of another string.	S1 = "aab", S2 = "axy", S3 = "aaxaby"	TRUE	
92	Check if two strings are pq-balanced.	S1 = "pqqp", S2 = "qpqp"	Example dependent on specific "pq-balanced" definition	
93	Match strings with wildcard characters ($*$, ?).	Pattern = "a?c", Text = "axcde"	TRUE	
94	Find the smallest window containing all characters of another string.	S1 = "ADOBECODEBANC", S2 = "ABC"	"BANC"	
95	Find the second most frequent character.	S = "aabbccdde"	c' or 'd'	
96	Find the second most frequent word.	S = "a b a c b"	c'	
97	Check if two given strings appear at the end of each other (ignoring case).	S1 = "abc", S2 = "Xabc"	TRUE	
98	Check if the first 'z' is immediately followed by another 'z'.	S1 = "zzyy", S2 = "zyzz"	S1: True, S2: False	
99	Check if a 'z' is happy (surrounded by same chars).	S = "azzb"	FALSE	
100	Return true if string contains 'abc' not followed by '.'.	S1 = "abcx", S2 = "abc."	S1: True, S2: False	

101	Check if a string is a valid palindrome ignoring spaces and punctuation.	S = "A man, a plan, a canal: Panama"	TRUE	
102	Reverse a string using recursion.	S = "abc"	"cba"	
103	Check if a string contains balanced parentheses.	S = "((()))"	TRUE	
104	Check if a string contains balanced brackets of all types (()), {}, []).	S = "{[()]}"	TRUE	
105	Find the longest valid parentheses substring.	S = "()(())"	6	
106	Generate all subsequences of a string.	S = "ab"	"", "a", "b", "ab"	
107	Check if a string is a pangram (contains every letter).	S = "The quick brown fox jumps over the lazy dog"	TRUE	
108	Check if a string is an isogram (no repeating letters).	S = "ambidextrous"	TRUE	
109	Find the lexicographically smallest substring of length k.	S = "banana", k = 3	"ana"	
110	Find the lexicographically largest substring of length k.	S = "banana", k = 3	"nan"	
111	Check if a string can be rearranged into a palindrome.	S = "aabbc"	TRUE	
112	Find the minimum number of deletions to make a string palindrome.	S = "aebcbda"	2 (delete 'e', 'd' → "abcba")	
113	Find the minimum number of insertions to make a string palindrome.	S = "aebcbda"	2 (insert 'd', 'e' → "adebcbeda")	
114	Check if one string is a subsequence of another.	S1 = "ace", S2 = "abcde"	TRUE	
115	Find the edit distance (Levenshtein distance) between two strings.	S1 = "kitten", S2 = "sitting"	3	
116	Check if a string is a valid shuffle of two other strings.	S1 = "xy", S2 = "12", S3 = "x1y2"	TRUE	
117	Check if a string contains duplicate substrings.	S = "ababa"	True (e.g., "aba")	
118	Find the longest repeated substring.	S = "banana"	"ana"	
119	Find the smallest substring containing all vowels.	S = "aeiouy"	"aeiou"	
120	Find the longest substring containing only vowels.	S = "abaeiouy"	"aeiou"	
121	Check if a string contains only binary digits (0/1).	S1 = "1010", S2 = "102"	S1: True, S2: False	
122	Convert a binary string to decimal.	S = "101"	5	
123	Convert a decimal number to binary string.	N = 5	"101"	
124	Find the longest common subsequence of two strings.	S1 = "AGGTAB", S2 = "GXTXAYB"	"GTAB"	
125	Find the shortest common supersequence of two strings.	S1 = "AGGTAB", S2 = "GXTXAYB"	"AGGXTXAYB"	
126	Print all anagrams of a string.	S = "cat"	"cat, cta, act, atc, tca, tac"	
127	Group words that are anagrams from an array of strings.	Arr = ["eat", "tea", "tan", "ate", "nat"]	[["eat", "tea", "ate"], ["tan", "nat"]]	
128	Check if a string follows a given pattern.	Pattern = "abba", S = "dog cat cat dog"	TRUE	
129	Find the smallest window containing all distinct characters.	S = "aabcbde"	"aabcbde"	
130	Find the maximum occurring word.	S = "a b a c a"	a'	
131	Check if a string is a valid email address.	S = "test@example.com"	TRUE	
132	Check if a string is a valid IP address.	S = "192.168.1.1"	TRUE	
133	Convert a Roman numeral string to integer.	S = "XIV"	14	
134	Convert an integer to Roman numeral string.	N = 14	"XIV"	
135	Check if two strings differ by exactly one character.	S1 = "pale", S2 = "ple"	False (differs by insertion/deletion)	
136	Check if two strings are one edit distance apart.	S1 = "pale", S2 = "ple"	TRUE	
137	Find the length of the longest substring with at most k distinct characters.	S = "eceba", k = 2	3 ("ece")	
138	Find all palindromic partitions of a string.	S = "aab"	["a", "a", "b"], ["aa", "b"]	
139	Check if a string can be segmented into valid dictionary words.	S = "applepenapple", Dict = ["apple", "pen"]	TRUE	
140	Implement KMP algorithm for substring search.	Text = "abcxabc", Pattern = "abc"	0, 4 (indices)	
141	Implement Rabin-Karp algorithm for substring search.	Text = "abcxabc", Pattern = "abc"	0, 4 (indices)	
142	Implement Z algorithm for substring search.	Text = "aabaaab", Pattern = "aab"	0, 4 (indices)	
143	Check if a string is valid JSON format (basic check).	S = '{\text{"key"}: \text{"value"}}'	TRUE	
144	Check if a string is valid HTML/XML tag sequence.	S = "<a>"	FALSE	
145	Remove HTML tags from a string.	S = " Title "	"Title"	
146	Compress a string using run-length encoding.	S = "aaabbc"	"a3b2c1"	
147	Decompress a run-length encoded string.	S = "a3b2c1"	"aaabbc"	

148	Find the first recurring substring of length k.	S = "abcab", k = 2	"ab"	
149	Check if two strings are scrambled versions of each other.	S1 = "great", S2 = "rgeat"	TRUE	
150	Find the lexicographically next permutation of a string.	S = "abc"	"acb"	
151	Find the lexicographically previous permutation of a string.	S = "acb"	"abc"	